

Robot Fleet Management Dashboard - Technical Assignment

Overview

Build a real-time robot fleet management dashboard that displays live robot statistics and health monitoring. This assessment evaluates your skills in **Node.js**, **React/Next.js**, **MongoDB**, and [uWebSockets.js](#).

We have provided sample code for the **backend** and **frontend**. Your task is to update this code to create the **Robot Fleet Management Dashboard** and **Robot Detail page**.

You are welcome to use any GenAI/Copilot tool to work on the project, as long as you can command and understand your whole codebase. You might be asked to provide reasoning for your design/strategy during the technical interview later.

Submission Requirements:

By the end of the project, please submit:

- The compilable source code
 - Provide a short README that explains how to run, test
 - Please push your source code to [github.com](#), **keep it private** and share it with hdang@symbolic.com, tnguyen@symbolic.com
 - Please commit your code regularly, at least once for each question.
- A video demonstration of your work
- Documentation
 - Put it into **README.md**
 - API documentation
 - Database schema explanation
 - Architecture overview and/or any technical considerations

Evaluation Criteria:

Your submission will be assessed based on:

- **Project completeness, with clear project structure and organization**
- **How modular/testable/extensible the codebase is**

Getting Started

- Download the repo [here](#).
- Follow the instructions (INSTRUCTIONS.md) to start the project, which includes:
 - Frontend
 - Backend and robot-simulator (simulate the robot connection for the test data)

Q1: Backend WebSocket Message Handler & Database Storage

This question handles the backend of the project, where incoming robot data is validated and processed before being broadcast to clients:

- Set up & install the necessary tools and dependencies.
- Parse incoming robot data from the simulator.
- Validate the data structure matches the expected format.
- Store each message in MongoDB with proper indexing.
- Broadcast real-time updates to connected dashboard clients.

Q2: Frontend Dashboard Page with Live Updates & Alerts

Build a dashboard with the following components:

Main Dashboard View

- Robot List Display: Show all robots along with their current status, indicating whether each robot is online or offline.
- Display key metrics: battery %, WiFi strength, charging status, temperature, and memory usage.
- Auto-refresh with live data.

Alert System

Implement two types of alerts:

- **Low Battery Alert:**
 - **Trigger:** Battery < 20% AND not charging
 - **Message:** "Robot {ID} is low battery!" (Warning)
 - **Behavior:** Notify once when the condition is met, reset when the battery ≥ 20% OR the robot starts charging
- **Critical Battery Alert:**
 - **Trigger:** Battery < 20% AND not charging for 5+ consecutive minutes
 - **Message:** "Robot {ID} will be shut down soon!" (Error)

- **Behavior:** Notify once when the 5-minute threshold is reached, reset when the battery $\geq 20\%$ OR the robot starts charging

Q3: Frontend Robot Detail Page with Historical Charts

The dashboard only shows live telemetry of the robots. We also need to implement a feature to show their historical data.

When the user clicks on a robot (a row or a card) on the dashboard page:

- Navigate to the detailed view for that specific robot.
- Display the robot's status, indicating whether it is online or offline.
- Display charts showing 6 hours of historical data for all metrics.
- Charts should update in real-time as new data arrives.

Q4: Scaling the Node.js Backend (Clustering & Horizontal Scaling)

Create backend/cluster.js that forks WORKERS (env) or CPU count max 4. Running “**npm run cluster**” starts multiple workers on the same port.

You are free to add another layer (optional) for scalability

Finally, please containerize the project with Docker Compose. Provide a compose.yml that defines Backend, Frontend, MongoDB, and additional services. Start the full stack with: **docker compose up -d**